

CS 546

Computer System Security

Lecture 5.1: Crypto (Integrity - II)

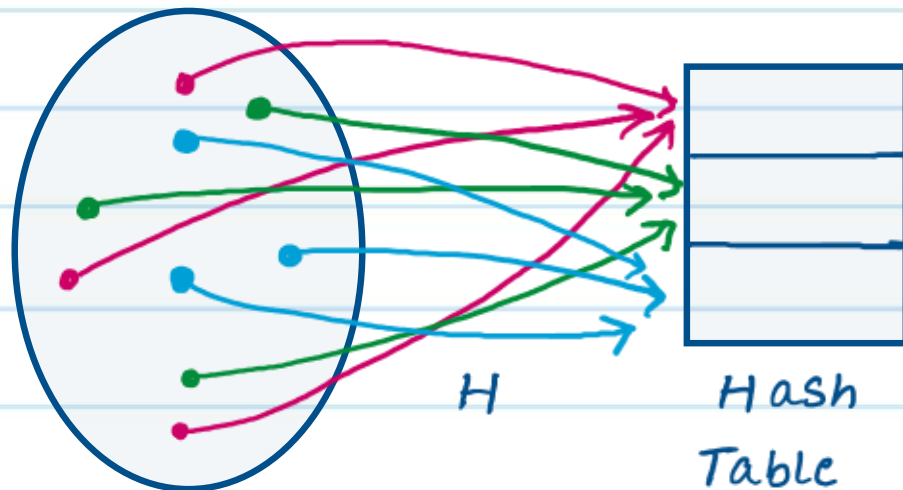
Instructor: Aarushi Goel

Fall 2026

Cryptographic Hash Functions

Hash Functions

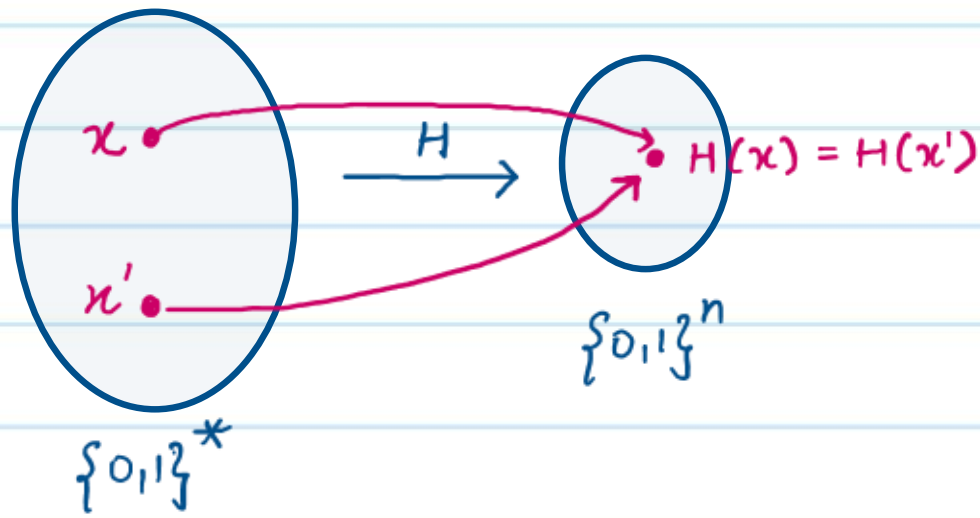
- * Functions that take inputs of arbitrary length and compress them into short, fixed length outputs.
- * $H: \{0,1\}^* \rightarrow \{0,1\}^n$
 ↓
 arbitrary length
- * Traditional use of hash functions was in data structures to enable faster lookup time when storing a set of elements.



To retrieve an element x it suffices to compute $H(x)$ and probe that row in the hash table to look for x . This is more efficient than looking up x in the original list.

Collisions in Hash Functions

- * A hash function naturally has *collisions*, where a collision is a pair of distinct elements x and x' , such that $H(x) = H(x')$.
- * A good hash function yields few collisions, because when a collision happens, two elements end up in the same row in the hash table, increasing the lookup time.



Collision - Resistant Hash Functions

- * Collision-resistant hash functions (CRHF) are cryptographic versions of hash functions, where the goal is not just to minimize the number of collisions, but it should also be infeasible for any probabilistic polynomial-time algorithm to find a collision in H .
- * Note that CRHFs are still compression functions, so collisions exist, but we just want such collisions to be hard to find.
- * We typically consider Keyed CRHFs.

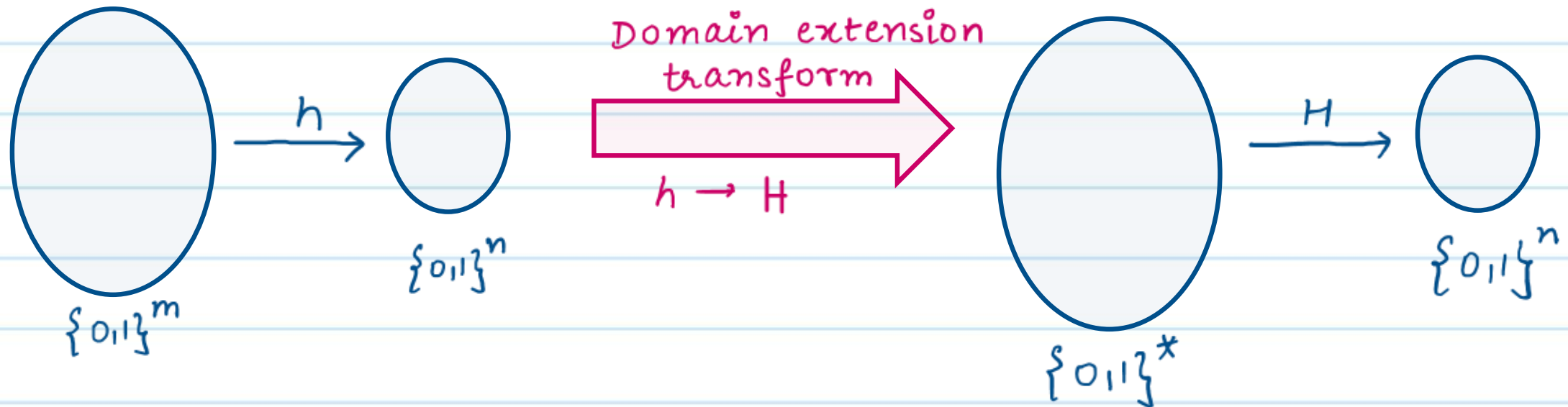
$$H_K(x) = H(K, x)$$

unlike all other primitives we have discussed so far, here we don't care about hiding the key.

- * It should be hard to find a collision in H^K , for randomly chosen K , even given K .

Domain Extension

- * Hash functions are often constructed by first designing a CRHF that handles fixed-length inputs, and then using domain extension to handle arbitrary-length inputs.



- * Merkle-Damgård transform is an example of domain extension.

Merkle-Damgård Transform

* Let $h_K: \{0,1\}^{2n} \rightarrow \{0,1\}^n$ be a CRHF

* We can design a CRHF $H_K: \{0,1\}^* \rightarrow \{0,1\}^n$ as follows:

1. Let $x \in \{0,1\}^L$ be an L -bit input that we want to hash using H .

2. Parse $x = x_1, \dots, x_B$, where $B = \frac{L}{n}$ and each $x_i \in \{0,1\}^n$.

If L is not divisible by n , we pad x with appropriately many 0s.

3. Set $IV = 0^n$ ^{initialization vector}.



Claim: If h_K is a CRHF, then so is H_K .

Proof:

Let us assume for the sake of contradiction that it is easy to find x, x' , such that $H_K(x) = H_K(x')$. We will show that it must then also be easy to find a collision in h_K , to arrive at a contradiction.

Case 1: $|x| \neq |x'|$: Let $x \in \{0,1\}^L$ and $x' \in \{0,1\}^{L'}$. If $H_K(x) = H_K(x')$, then it must be the case that $h_K(\cdot, L) = h_K(\cdot, L')$. This is a collision on h_K . This means, an adversary who can find collision in H_K , can also find a collision in h_K . But since h_K is a CRHF, it should be hard for any PPT adversary to find such collisions. Hence, it must also be hard for any PPT adversary to find collisions in H_K .

Case 2: $|x| = |x'|$: Let $x, x' \in \{0,1\}^L$. Parse $x = x_1, \dots, x_B$ & $x' = x'_1, \dots, x'_B$.

Since $x \neq x'$, there must exist at least one $i \in [B]$, such that $x_i \neq x'_i$. Let's pick the largest such i .

If $H_K(x) = H_K(x')$, then it must be the case that $h_K(\cdot, x_i) = h_K(\cdot, x'_i)$.

This is a valid collision in h_K . But since h_K is a CRHF, it should be hard for any PPT adversary to find such collisions.

Hence, it must also be hard for any PPT adversary to find collisions in H_K .

$\Rightarrow H_K$ is a CRHF.

Applications of CRHFs

- * **Password Authentication:** Instead of storing plaintext passwords on servers, websites can store a hash of the passwords instead. This ensures that the passwords are not compromised even if the server is compromised.
- * **Deduplicate Files:** To avoid storing duplicate copies of files, we can use hashes of files as identifiers. We can hash two files to produce identifiers h_1 and h_2 . If $h_1 \neq h_2$, this implies $D_1 \neq D_2$. If $h_1 = h_2$, it almost always implies file $D_1 = \text{file } D_2$.

* **HMAC:** Hash functions can be used as MACs.

$MAC_K(m) = H(K\|m)$. It is computationally difficult to find another $K\|m'$ that produces the same hash.

* However, when using a Merkle-Damgård based hash, an adversary could potentially attach some additional s to m to get $m' = m\|s$ such that they can easily compute $tag' = H(tag\|s)$

* Therefore, in practice, we use a nested MAC like

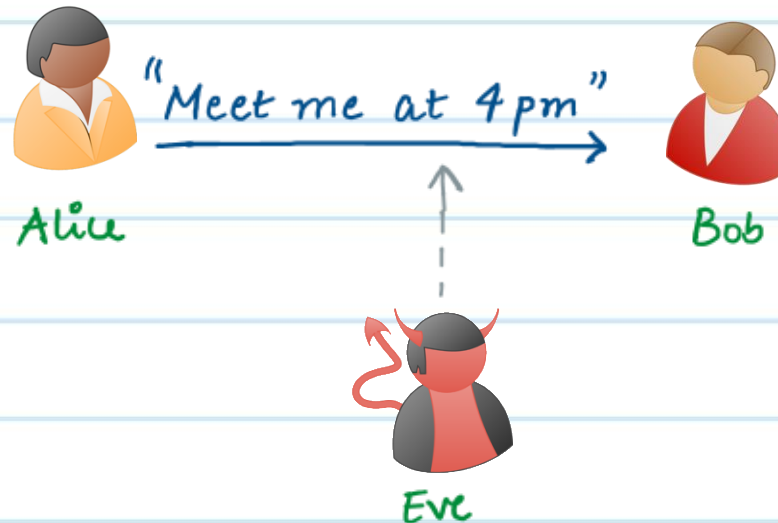
$$MAC_K(m) = H(K\|H(K\|m)).$$

* **Hash-and-MAC:** If we have a MAC scheme for signing fixed length messages, and want to use it for signing arbitrary-length messages, we can hash the message to appropriate length digest and then sign the digest.

Message Authentication Codes (MACs)

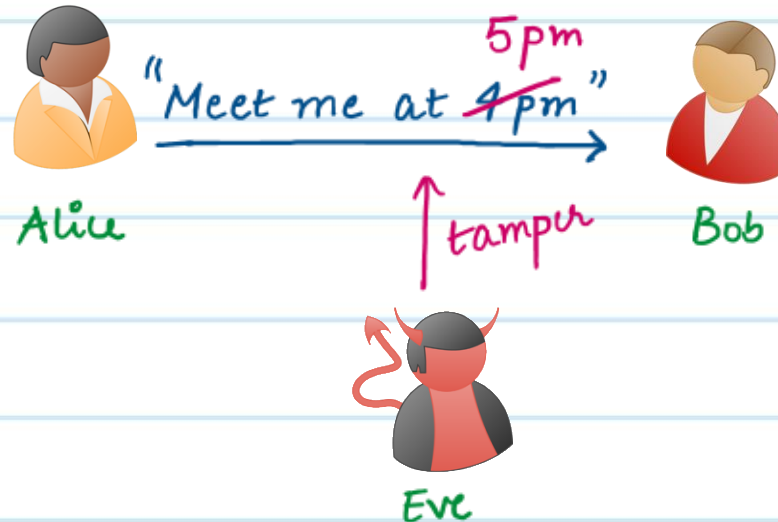
Private Communication

- * So far we have considered the problem of private communication, where Alice wants to send a secret message to Bob, while making sure **Eve** who might be eavesdropping on their communication.
- * Encryption helps with this.
- * But an Eve who is only listening to a private communication, is still a pretty "well-behaved" adversary.



Tamper-Proof Communication

- * In reality, adversaries could be much worse.
- * What if the adversary can not only eavesdrop on a communication channel, but also *tamper* with messages on this communication channel.
- * Does encryption naturally help prevent this? **No!**



Message Integrity in the Non-Digital World

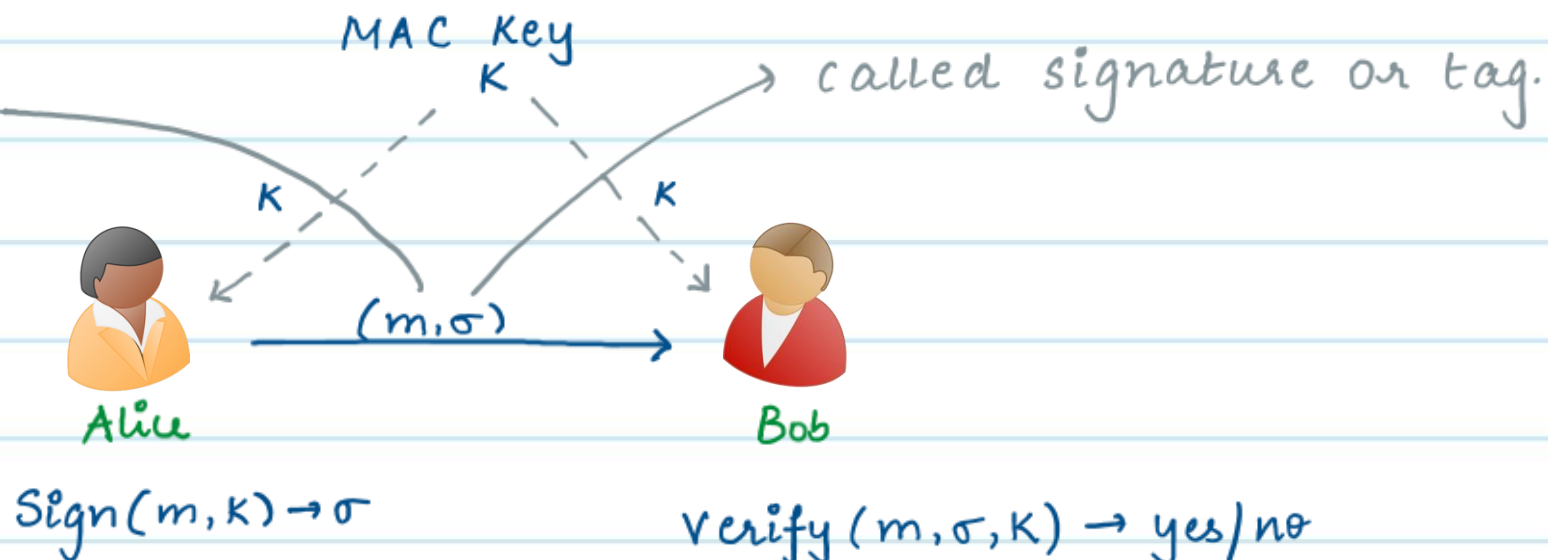
- * In the physical world, we use signatures on documents to ensure unauthorised entities cannot tamper with them, without forging the signature, which is hard to do.
- * We need something similar in the digital world.



Message Authentication Codes (MACs)

* Observe that our

goal here is not to hide m . Here, we are just focusing on message integrity.



* Recall from the last lecture that we combined encryption and MAC in ***authenticated encryption*** to ensure integrity of encrypted messages.



Adversary

* If the adversary tries to change $m \rightarrow m'$, they will also have to find a corresponding σ' , such that $\text{Verify}(m, \sigma, K) \rightarrow \text{yes}$. This should be hard if they don't have K .

Message Authentication Codes (MACs)

Defined by three algorithms:

- * $\text{Keygen} \xrightarrow{\$} K$
- * $\text{Sign}(K, m) \rightarrow \sigma$
- * $\text{Verify}(K, m, \sigma) \rightarrow \text{yes / no}$

→ **Correctness:** Let $\text{Keygen} \xrightarrow{\$} K$, then $\forall m$
 $\Pr[\text{Verify}(K, \text{Sign}(K, m), m)] = 1$

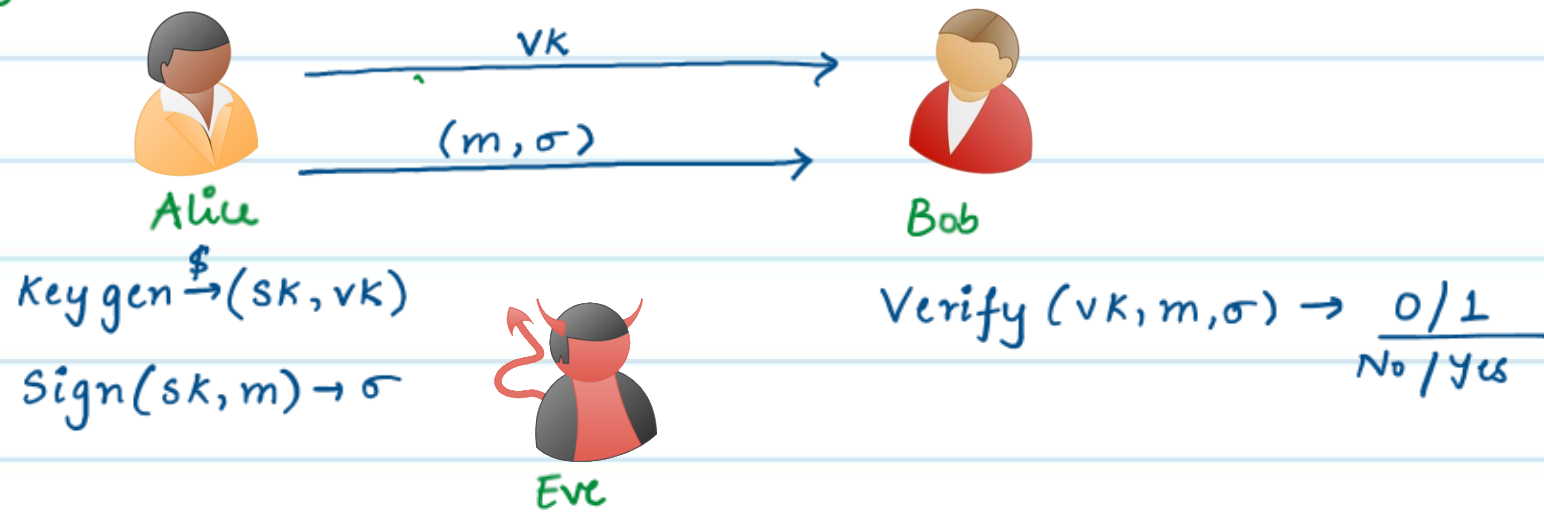
→ **Unforgeability:** A PPT adversary who does not have the key K , cannot forge a valid MAC on any message of its choice, even if it sees a MAC on some message.

Digital Signatures

Digital Signatures

- * Digital Signatures are essentially public-key variants of MACs.
- * Instead of a single key, in digital signatures, we have two separate keys: one for **signing**, and another one for **verification**.
- * The ***signing key*** is like a private-key
- * The ***verification key*** is like a public-key, and need not be kept hidden.
- * The unforgeability guarantee is also only defined w.r.t., the signing key. Any PPT adversary who does have the signing key, should not be able to forge signatures on messages of their choice.

Digital Signatures



* **Correctness**: Let $\text{Keygen} \xrightarrow{\$} (sk, vk)$, then $\forall m$,
 $\Pr[\text{Verify}(vk, m, \text{Sign}(sk, m)) \rightarrow 1] = 1$

* **Unforgeability**: Any PPT adversary who does not have sk , should not be able to forge valid signatures on messages of its choice, even if it has vk & sees many valid signatures signed using sk .

Basic Number Theory

* $\gcd(a, b)$: greatest common divisor.

Let a, b both be of n -bits. Their $\gcd$ can be computed in $O(n)$ time.

Euclidean Algorithm: $\gcd(17, 12) = 1$

$$17 \bmod 12 = 5$$

$$12 \bmod 5 = 2$$

$$5 \bmod 2 = 1$$

$$2 \bmod 1 = 0$$

$\gcd(18, 12) = 6$

$$18 \bmod 12 = 6$$

$$12 \bmod 6 = 0$$

* $\gcd(a, N) = 1$: a & N are co-prime.

$\Rightarrow \exists b$ s.t. $a \cdot b \equiv 1 \pmod{N}$: a is invertible modulo N , $b = a^{-1}$ is its inverse.

How to compute b ?

Extended Euclidean Alg

$$* \mathbb{Z}_N^* = \{a \mid a \in [1, N-1], \gcd(a, N) = 1\}$$

Euler's phi (totient) function $\phi(N) := |\mathbb{Z}_N^*|$

eg. if N is prime, $\phi(N) = N-1$

$$\begin{aligned} \text{if } N = p \cdot q \text{ (} p, q \text{ are primes), } \phi(N) &= (p-1) \cdot (q-1) \\ &= (N-1) - (p-1) - (q-1) \end{aligned}$$

* Euler's Theorem: $\forall a, N$ where $\gcd(a, N) = 1$, $a^{\phi(N)} \equiv 1 \pmod{N}$

Corollary: If $d \equiv e^{-1} \pmod{\phi(N)}$, then $(a^d)^e \equiv a \pmod{N}$

$\Downarrow$

$$d \cdot e \equiv 1 \pmod{\phi(N)}$$

$\Downarrow$

$$d \cdot e = \phi(N) \cdot c + 1$$

$\Downarrow$

$$(a)^{de} \equiv (a)^{\phi(N) \cdot c + 1} \pmod{N}$$

$$\equiv (a)^{\phi(N) \cdot c} \times (a)^1 \pmod{N}$$

$$\equiv 1^c \cdot a \pmod{N}$$

$$\equiv a \pmod{N}$$

→ Rivest, Shamir, Adleman

RSA Assumption

- * **Factoring Assumption:** Generate two n -bit primes p, q ($p \neq q$). Compute $N = p \cdot q$. Given N , it is computationally hard to find p & q (classically).
- * **RSA Assumption:** Generate two n -bit primes p, q ($p \neq q$). Compute $N = p \cdot q$, $\phi(N) = (p-1)(q-1)$. Choose e s.t., $\gcd(e, \phi(N)) = 1$. Compute $d \equiv e^{-1} \pmod{\phi(N)}$. Given (N, e) & a random $y \xleftarrow{\$} \mathbb{Z}_N^*$, it is computationally hard to find x , s.t., $x^e \equiv y \pmod{N}$.

If you can break the factoring assumption, then you can also break the RSA assumption.

Textbook

RSA-based Digital Signature Scheme

* Keygen: Generate two n -bit primes p, q ($p \neq q$). Compute $N = p \cdot q$
 $\phi(N) = (p-1)(q-1)$. Choose e s.t., $\gcd(e, \phi(N)) = 1$.
Compute $d \equiv e^{-1} \pmod{\phi(N)}$.
 $SK = d$ $VK = (N, e)$

* Sign(SK, m): $\sigma \equiv m^d \pmod{N}$

* Verify(VK, m, σ): Check if $\sigma^e \equiv m \pmod{N}$, if so, output 1,
else, output 0.

Is this unforgeable?? No.

Textbook

RSA-based Digital Signature Scheme

I This construction is malleable.

* Given two signatures $\sigma_1 = m_1^d \pmod{N}$, $\sigma_2 = m_2^d \pmod{N}$, an adversary can compute $\sigma_1 \cdot \sigma_2 \pmod{N}$, and this will be a valid signature on $m_1 \cdot m_2$. Therefore this is not a secure signature scheme. In a secure (unforgeable) signature scheme, it should not be possible for a PPT adversary to guess a valid signature on $m_1 \cdot m_2$ after seeing valid signatures on m_1 & m_2 .

II Another issue with this scheme.

* An adversary can first choose an arbitrary σ & then compute the corresponding message $m \equiv \sigma^e \pmod{N}$. Now (m, σ) is a valid message, signature pair. Note that in this attack, even though m may be arbitrary & not what adversary wanted to choose, in a secure signature scheme, we don't want adv to forge signatures on *ANY* message.

(Fixed-Version)

RSA-based Digital Signature Scheme

- * Keygen: Generate two n -bit primes p, q ($p \neq q$). Compute $N = p \cdot q$
 $\phi(N) = (p-1)(q-1)$. Choose e s.t., $\gcd(e, \phi(N)) = 1$.
Compute $d \equiv e^{-1} \pmod{\phi(N)}$.

$SK = d$

$VK = (N, e)$

- * Sign(SK, m): $\sigma \equiv H(m)^d \pmod{N}$

- * Verify(VK, m, σ): Check if $\sigma^e \equiv H(m) \pmod{N}$, if so, output 1,
else, output 0.